



Apex Sports Club Presentation Script

What to say on every slide — a presenter's guide to the 21-slide deck

Deck: Apex_Sports_Club_Presentation.pptx / Apex_Sports_Club_Presentation_Simple.pptx (21 slides, 16:9)

Time: roughly 12–15 minutes · **Mode:** Presenter View (Alt+F5) to read these notes while the audience sees the slide

Tip: say what is on the slide first, add one fact, then hand off to the next slide.

- | | |
|---|---|
| 01 Apex Sports Club | 12 Data Flow Diagram — Current System |
| 02 What we will cover | 13 Data Flow Diagram — Proposed System (Level 0) |
| 03 Project Overview | 14 Data Flow Diagram — Proposed System (Level 1) |
| 04 Technology Stack | 15 Database Design — ERD |
| 05 System Architecture | 16 Key Features |
| 06 Member Login | 17 Database Schema |
| 07 Member Registration | 18 Payments & Integrations |
| 08 Admin Login & Dashboard | 19 Security Best Practices |
| 09 Use Case Diagram — Members | 20 Setup & Documentation Rules |
| 10 Use Case Diagram — Administrators | 21 Thank You |
| 11 System Analysis | |

WHAT TO SAY

Good morning/afternoon everyone. I'm presenting Apex Sports Club — a web application that manages members, bookings, league teams, fixtures, standings, payments and club admin operations. It's built with vanilla PHP, MySQL and Bootstrap 5, so it runs locally on XAMPP and on any PHP hosting.

WHAT TO SAY

Here's the roadmap: we start with an overview and the tech stack, look at the system architecture, walk through the member and admin portals, review the use-case diagrams and database schema, then finish with integrations, security and how to get it running. Roughly 10 minutes total.

WHAT TO SAY

Apex Sports Club is a single platform for the whole club. Members get a self-service portal — register, log in, update their profile, book facilities and join teams. Club staff get a restricted dashboard to manage every entity. Two audiences, one system.

WHAT TO SAY

We deliberately kept the stack simple and proven: PHP 7.4+ on the backend, MySQL for the database, Bootstrap 5 with vanilla JavaScript on the front, and Apache or Nginx as the server. No Composer, no heavy framework — easy to audit and easy to deploy.

WHAT TO SAY

The flow is straightforward: a member or admin browser hits the web server, which serves the PHP application split into public, admin, includes and config folders, backed by the MySQL database. On the side we integrate Paystack for payments, Brevo for email and API-SPORTS for optional data.

WHAT TO SAY

This is the member login at `public/login.php`. Members sign in with email and password; sessions handle authentication and hashed credentials keep passwords safe. A 'remember me' option persists the session, and new members can jump straight to registration from the same screen.

WHAT TO SAY

New members create their account at `public/register.php`. We collect name, email and phone, confirm the password twice to catch typos, and optionally capture a sport interest to help admins plan sessions. CAPTCHA protection can be switched on via the `.env` file.

WHAT TO SAY

Staff reach the control panel at `admin/admin_login.php`. The seeded demo account is `admin@sportsclub.com / admin123` — this must be changed before any production use. Once inside, admins manage members, sports, coaches, facilities, equipment, bookings, payments, leagues, teams, fixtures and standings.

WHAT TO SAY

This UML use-case diagram shows what a club member can do inside the public portal: register, log in, update their profile, browse sports and coaches, book facilities, view their bookings, join league teams, check fixtures and standings, and make payments.

WHAT TO SAY

Here the actor is the administrator, with use cases inside the admin panel boundary — managing every entity the club tracks, from members and coaches through bookings and payments to leagues, fixtures and standings.

WHAT TO SAY

This slide opens the analysis chapter. I will walk through three data flow diagrams: the current system at Level 0, the proposed system at Level 0, and the proposed system broken into six sub-processes at Level 1.

Data Flow Diagram — Current System

12

WHAT TO SAY

The current system runs on paper. A member submits a form to the club clerk, who records the booking and collects cash, with the finance office keeping the ledger. Nothing is validated automatically — that is exactly the problem the system solves.

Data Flow Diagram — Proposed System (Level 0)

13

WHAT TO SAY

Here is the proposed system as a single process with four external parties: the member, the administrator, the Paystack gateway and the Brevo email service. Login, booking, approval, payment and receipt email all flow through the one system boundary.

Data Flow Diagram — Proposed System (Level 1)

14

WHAT TO SAY

Level 1 breaks the system into six sub-processes — registration, login and profile, facility booking, payment processing, leagues and teams, and fixtures and standings. Each one reads and writes its own data store, D1 to D6.

WHAT TO SAY

This entity relationship diagram shows the eight core tables and how they link. Members own bookings, bookings own payments, and sports connect to leagues, teams, fixtures and standings — every relationship is one-to-many through primary and foreign keys.

WHAT TO SAY

Eight features cover the day-to-day: member self-service, facility and session booking with admin approval, multi-sport league and team registration, automatic standings recalculation, Paystack payments, Brevo email receipts, CAPTCHA protection, and the full admin control panel.

WHAT TO SAY

The data lives in three groups. Core tables handle members, sports, coaches, facilities, equipment, bookings and payments. The league module adds leagues, teams and team memberships. Competition tables cover fixtures and standings, with standings recalculated automatically from results.

WHAT TO SAY

All external services are wired through the `.env` file. Paystack initializes checkout and verifies it in the callback before recording a payment. Brevo sends email receipts, no CAPTCHA is enforced, and API-SPORTS is an optional data source. Remember: commit `.env.example`, never `.env`.

WHAT TO SAY

We follow strict rules: secrets live only in .env, the default admin login must change before production, payment callbacks run over HTTPS, and the Paystack secret key stays private. Passwords are hashed and sessions gate protected pages. Stronger validation and CSRF protection are recommended next steps.

WHAT TO SAY

Getting it running is quick: drop the project into the web root, start Apache and MySQL, import the three SQL files in order, copy `.env.example` to `.env` and configure the keys, then open the app. `DOCUMENTATION.md` covers all of this plus deployment and troubleshooting.

WHAT TO SAY

That wraps up the presentation. Thank you for your time — I'm happy to take questions, or walk through the live application at [public/index.php](#) and the admin panel at [admin/admin_login.php](#).